

Python으로 TurtleBot3 자율주행해보기



Contents

- 1 [SBC] 먼저 원격 접속후 로봇을 실행
- 2 [PC] PC에서 내비게이션을 실행
- 3 실물 로봇의 위치와 맵 위에서의 위치를 일치시켜주자
- 4 [PC] 그리고 새 터미널을 열어서 환경을 부르고 주피터 노트북 실행
- 5 nav2 클래스 불러오기
- 6 nav2를 초기화 하고
- 7 목표 좌표를 설정 하기
- 8 목표 좌표로 보내기
- 9 주행 중에 상황 피드백 받기
- 10 주행 결과 확인하기
- 11 한번에 goal좌표를 여러개 보내기위해 리스트 만들기
- 12 follow waypoint 실행하기
- 13 현재 주행 중인 상황 피드백 확인하기
- 14 nav2 종료하고 ros 연결 종료하기

Python으로 TurtleBot3 자율주행해보기

Exported from Confluence · <https://pinkwink.atlassian.net>

[SBC] 먼저 원격 접속후 로봇을 실행

Code

```
1 ros2 launch turtlebot3_bringup robot.launch.py
```

```
ubuntu@ubuntu:~$ ros2 launch turtlebot3_bringup robot.launch.py
[INFO] [launch]: All log files can be found below /home/ubuntu/.ros/log/2024-08-07-07-10-53-950951-ubuntu-1745
[INFO] [launch]: Default logging verbosity is set to INFO
urdf_file_name : turtlebot3_waffle_pi.urdf
[INFO] [robot_state_publisher-1]: process started with pid [1746]
[INFO] [ld08_driver-2]: process started with pid [1748]
[INFO] [turtlebot3_ros-3]: process started with pid [1750]
[turtlebot3_ros-3] [INFO] [1723014654.535771708] [turtlebot3_node]: Init TurtleBot3 Node Main
[turtlebot3_ros-3] [INFO] [1723014654.540175686] [turtlebot3_node]: Init DynamixelSDKWrapper
[turtlebot3_ros-3] [INFO] [1723014654.543057223] [DynamixelSDKWrapper]: Succeeded to open the port(/dev/ttyACM0)!
[turtlebot3_ros-3] [INFO] [1723014654.551202478] [DynamixelSDKWrapper]: Succeeded to change the baudrate!
[robot_state_publisher-1] [INFO] [1723014654.559478044] [robot_state_publisher]: got segment base_footprint
[robot_state_publisher-1] [INFO] [1723014654.559766235] [robot_state_publisher]: got segment base_link
[robot_state_publisher-1] [INFO] [1723014654.559816059] [robot_state_publisher]: got segment base_scan
[robot_state_publisher-1] [INFO] [1723014654.559846554] [robot_state_publisher]: got segment camera_link
[robot_state_publisher-1] [INFO] [1723014654.559872383] [robot_state_publisher]: got segment camera_rgb_frame
[robot_state_publisher-1] [INFO] [1723014654.559899137] [robot_state_publisher]: got segment camera_rgb_optical_frame
[robot_state_publisher-1] [INFO] [1723014654.559926077] [robot_state_publisher]: got segment caster_back_left_link
[robot_state_publisher-1] [INFO] [1723014654.559951591] [robot_state_publisher]: got segment caster_back_right_link
[robot_state_publisher-1] [INFO] [1723014654.559976402] [robot_state_publisher]: got segment imu_link
[robot_state_publisher-1] [INFO] [1723014654.560001323] [robot_state_publisher]: got segment wheel_left_link
[robot_state_publisher-1] [INFO] [1723014654.560026486] [robot_state_publisher]: got segment wheel_right_link
```

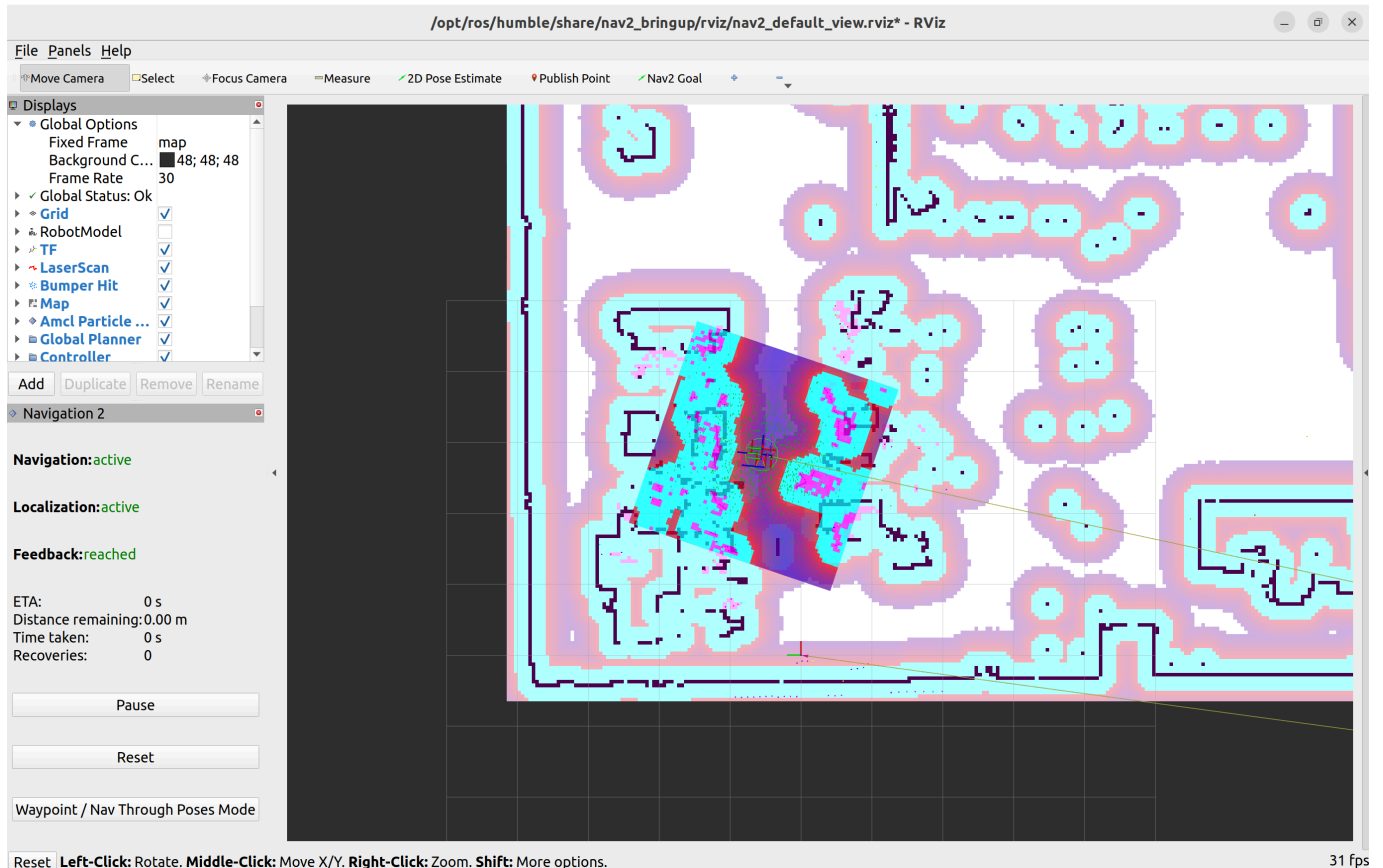
[PC] PC에서 내비게이션을 실행

Code

```
1 ros2 launch turtlebot3_navigation2 navigation2.launch.py map:=<map의 yaml파일 주소>
```

```
mkh@mkh:~$ ros2 launch turtlebot3_navigation2 navigation2.launch.py map:=pinklab.yaml
[INFO] [launch]: All log files can be found below /home/mkh/.ros/log/2024-08-07-16-12-25-071800-mkh-37218
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [component_container_isolated-1]: process started with pid [37231]
[INFO] [rviz2-2]: process started with pid [37233]
[component_container_isolated-1] [INFO] [1723014746.222554225] [nav2_container]: Load Library: /opt/ros/humble/lib/libmap_server_core.so
[component_container_isolated-1] [INFO] [1723014746.230458421] [nav2_container]: Found class: rclcpp_components::NodeFactoryTemplate<nav2_map_server::CostmapFilterInfoServer>
[component_container_isolated-1] [INFO] [1723014746.230481628] [nav2_container]: Found class: rclcpp_components::NodeFactoryTemplate<nav2_map_server::MapSaver>
[component_container_isolated-1] [INFO] [1723014746.230486089] [nav2_container]: Found class: rclcpp_components::NodeFactoryTemplate<nav2_map_server::MapServer>
[component_container_isolated-1] [INFO] [1723014746.230488874] [nav2_container]: Instanti
```

실물 로봇의 위치와 맵 위에서의 위치를 일치시켜주자



[PC] 그리고 새 터미널을 열어서 환경을 부르고 주피터 노트북 실행

```
mkh@mkh:~$ jupyter notebook
[I 2024-08-07 16:14:41.788 ServerApp] jupyter_lsp | extension was successfully linked.
[I 2024-08-07 16:14:41.791 ServerApp] jupyter_server_terminals | extension was successfully linked.
[I 2024-08-07 16:14:41.793 ServerApp] jupyterlab | extension was successfully linked.
[W 2024-08-07 16:14:41.794 JupyterNotebookApp] 'ip' has moved from NotebookApp to ServerApp. This config will be passed to ServerApp. Be sure to update your config before our next release.
[W 2024-08-07 16:14:41.794 JupyterNotebookApp] 'port' has moved from NotebookApp to ServerApp. This config will be passed to ServerApp. Be sure to update your config before our next release.
[W 2024-08-07 16:14:41.794 JupyterNotebookApp] 'port' has moved from NotebookApp to ServerApp. This config will be passed to ServerApp. Be sure to update your config before our next release.
```

nav2 클래스 불러오기

```
In [1]: from nav2_simple_commander.robot_navigator import BasicNavigator
import rclpy

rclpy.init()
nav = BasicNavigator()
```

nav2를 초기화 하고

```
In [2]: nav.waitUntilNav2Active()
```

```
[INFO] [1706786996.260341001] [basic_navigator]: Nav2 is ready for use!
```

목표 좌표를 설정 하기

목표 좌표는 rviz2에서 Publish Point로 알아보기

```
In [5]: from geometry_msgs.msg import PoseStamped

goal_pose = PoseStamped()
goal_pose.header.frame_id = 'map'
goal_pose.header.stamp = nav.get_clock().now().to_msg()
goal_pose.pose.position.x = 1.5
goal_pose.pose.position.y = 0.55
goal_pose.pose.orientation.w = 0.707
goal_pose.pose.orientation.z = 0.707
```

목표 좌표로 보내기

```
In [6]: nav.goToPose(goal_pose)
```

```
[INFO] [1706787259.553513934] [basic_navigator]: Navigating to goal: 1.5 0.55...
```

```
Out[6]: True
```

주행 중에 상황 피드백 받기

주행 중에 실행할 것

```
[12]: from rclpy.duration import Duration

i = 0
while not nav.isTaskComplete():
    i = i + 1
    feedback = nav.getFeedback()
    if feedback and i % 5 == 0:
        print('Distance remaining: ' + '{:.2f}'.format(
            feedback.distance_remaining) + ' meters.')

    if Duration.from_msg(feedback.navigation_time) > Duration(seconds=10.0):
        nav.cancelTask()
```

```
Distance remaining: 1.45 meters.
Distance remaining: 1.37 meters.
Distance remaining: 1.26 meters.
Distance remaining: 1.07 meters.
Distance remaining: 0.95 meters.
Distance remaining: 0.85 meters.
Distance remaining: 0.75 meters.
Distance remaining: 0.62 meters.
Distance remaining: 0.50 meters.
Distance remaining: 0.41 meters.
```

주행 결과 확인하기

```
In [13]: from nav2_simple_commander.robot_navigator import TaskResult

result = nav.getResult()
if result == TaskResult.SUCCEEDED:
    print('Goal succeeded!')
elif result == TaskResult.CANCELED:
    print('Goal was canceled!')
elif result == TaskResult.FAILED:
    print('Goal failed!')
else:
    print('Goal has an invalid return status!')
```

Goal succeeded!

한번에 goal좌표를 여러개 보내기위해 리스트 만들기

```
In [10]: goal_pose_list = []
goal_pose1 = PoseStamped()
goal_pose1.header.frame_id = 'map'
goal_pose1.header.stamp = nav.get_clock().now().to_msg()
goal_pose1.pose.position.x = 1.5
goal_pose1.pose.position.y = 0.55
goal_pose1.pose.orientation.w = 0.707
goal_pose1.pose.orientation.z = 0.707
goal_pose_list.append(goal_pose1)

goal_pose2 = PoseStamped()
goal_pose2.header.frame_id = 'map'
goal_pose2.header.stamp = nav.get_clock().now().to_msg()
goal_pose2.pose.position.x = 1.5
goal_pose2.pose.position.y = -3.75
goal_pose2.pose.orientation.w = 0.707
goal_pose2.pose.orientation.z = 0.707
goal_pose_list.append(goal_pose2)

goal_pose3 = PoseStamped()
goal_pose3.header.frame_id = 'map'
goal_pose3.header.stamp = nav.get_clock().now().to_msg()
goal_pose3.pose.position.x = -3.6
goal_pose3.pose.position.y = -4.75
goal_pose3.pose.orientation.w = 0.707
goal_pose3.pose.orientation.z = 0.707
goal_pose_list.append(goal_pose3)
```

follow waypoint 실행하기

```
In [11]: nav_start = nav.get_clock().now()
nav.followWaypoints(goal_pose_list)

[INFO] [1706787608.206456332] [basic_navigator]: Following 3 goals....

Out[11]: True
```

현재 주행 중인 상황 피드백 확인하기

waypoint 주행중에 실행하기

```

from rclpy.duration import Duration

i = 0
while not nav.isTaskComplete():
    i = i + 1
    feedback = nav.getFeedback()
    if feedback and i % 5 == 0:
        print(
            'Executing current waypoint: '
            + str(feedback.current_waypoint + 1)
            + '/'
            + str(len(goal_pose_list))
        )
    now = nav.get_clock().now()

    if now - nav_start > Duration(seconds=600.0):
        navigator.cancelTask()

```

Executing current waypoint: 1/3
 Executing current waypoint: 1/3

nav2 종료하고 ros 연결 종료하기

In [34]: `nav.lifecycleShutdown()`

```

[INFO] [1689682029.171973083] [basic_navigator]: Shutting down lifecycle nodes based on lifecycle_manager.
[INFO] [1689682029.173491077] [basic_navigator]: Shutting down /lifecycle_manager_localization/managed_nodes
[INFO] [1689682029.468932357] [basic_navigator]: Shutting down /lifecycle_manager_navigation/managed_nodes

```

In [35]: `rclpy.shutdown()`



PinkLAB YouTube Channel

PinkLAB Studio — Robotics, AI, ROS2 and more!

 **Subscribe on YouTube**

https://www.youtube.com/@pinklab_studio